A Global Cross-Learning Demand Forecasting Engine for SMB Distribution

Direct multi-horizon estimation, conformal lead-time uncertainty, and cost-asymmetric model selection

Angel Zeledon Fernandez
Faro — *ForecastingCore* technical report

ABSTRACT

We describe the forecasting engine behind Faro, an inventory purchasing system for small and mid-sized distributors in Latin America. The operating regime is adversarial to classical per-series forecasting: catalogues of thousands of SKUs, median histories measured in months rather than years, intermittent and censored demand, and a decision whose loss function is sharply asymmetric. We set out the engine's construction in full: the canonical data contract and the exact feature algebra; recovery of stockout-censored demand; a global cross-learning gradient-boosted model that shares structure across the catalogue through per-series normalisation and series-identity conditioning; a direct multi-horizon formulation that removes recursive error compounding; split-conformal prediction bands calibrated per horizon and per lead time; and a selection rule that ranks candidate models by asymmetric cost rather than symmetric error. We close with the mapping from a predictive distribution to a purchase order, and with an explicit account of what the design does not solve.

1. Problem setting and notation

Let S denote the set of series in a tenant's catalogue. A series is a (SKU, location) pair; when a dataset carries no location dimension the engine assigns a single synthetic location so that the series key is uniform across the system. Time is discretised into buckets indexed by t, where a bucket is a day, a week, a fortnight or a month according to the granularity the session was trained at.

$$y_{s,t} \in \mathbb{R}_{\geq 0}, \qquad s \in S,\ t = 1,\ \ldots,\ n_s \tag{1}$$

The quantity of interest is not y itself. Purchasing decisions are exposed to the demand that accumulates while an order is in transit, so the target functional is the distribution of the sum over the lead time L, evaluated at a service level β:

$$D_{s,T}(L) = \sum_{k=1}^{L} y_{s,T+k}, \qquad \mathrm{ROP}_s = Q_\beta[D_{s,T}(L)] \tag{2}$$

This framing matters more than any modelling choice that follows. A system that optimises the conditional mean of y and then bolts a Gaussian safety-stock formula onto it is answering a different question from the one the buyer asked. Sections 6 and 9 return to this point.

1.1 Information sets and the leakage boundary

Write $\mathscr{E}_{s,t}$ for everything observable about series s up to and including bucket t. Every feature the engine constructs for a prediction of $y_{s,t+h}$ is a measurable function of $\mathscr{E}_{s,t}$ together with deterministic calendar information about the target date. The engine enforces this by construction rather than by review: all autoregressive features are built from a shifted series, and the validation protocol withholds a gap of buckets before every evaluation window.

2. The canonical data contract

Uploads arrive with arbitrary column names in arbitrary languages. A mapping step resolves the user's columns onto a fixed canonical schema; unmapped optional fields are filled with declared defaults. Three fields are required and the rest are optional, but the optional ones are not decorative — each unlocks a specific capability, and the engine degrades explicitly rather than silently when they are absent.

Canonical field	Required	Role in the engine
sku	yes	Series identity; categorical conditioning variable c_s in the global model.
date	yes	Bucket index; source of all calendar features.
demand	yes	Target y.
store	no	Second identity dimension; makes the series key (SKU, location).
inventory	no	End-of-bucket stock. The *only* field that can distinguish "sold none" from "had none to sell"; gates Section 4.
price, regular_price, promo_price, discount	no	Exogenous regressors; also the basis of promotional indicators.
promo, promo_type	no	Event indicators.
lead_time	no	Per-SKU replenishment delay L used by Section 9.
cost	no	Unit cost; enters the holding/stockout cost ratio.

Table 1. The canonical schema. Defaults for unmapped optional fields are broadcast constants, which is itself a hazard: see Section 4.3.

3. Preprocessing

3.1 Transaction collapse

ERP exports commonly emit one row per sale rather than one row per bucket. Left unaggregated, three transactions of 4, 3 and 3 units on one day become three observations of a series whose level is then estimated near 3 instead of 10. The engine sums the target within each (date, series) key and carries non-additive columns forward by first value:

$$y_{s,t} = \sum_{r \in \mathcal{R}_{s,t}} q_r \tag{3}$$

where $\mathcal{R}_{s,t}$ is the set of raw rows sharing that key.

3.2 Gap filling and outlier treatment

Missing buckets are reindexed onto a regular grid at the detected native frequency, with the fill policy chosen by the user (zero, mean, forward, linear interpolation, or leave). Reindexing is abandoned for a series whose implied span exceeds a hard cap, since a single mistyped date would otherwise materialise millions of rows. Outliers are treated per series by a configurable rule: σ-clipping, percentile winsorisation, an IQR fence, removal with interpolation, or a log1p variance-stabilising transform.

$$y^{\text{clip}}_{s,t} = \min(\max(y_{s,t},\ Q_1 - k\,\text{IQR}),\ Q_3 + k\,\text{IQR}) \tag{4}$$

4. Censored demand recovery

A sales file records what was *sold*. On a bucket in which the SKU was out of stock, what was sold is a lower bound on what was demanded, and on a bucket that was stocked out from open to close the recorded zero means "could not sell", not "nobody wanted it". Formally the engine observes

$$y^{\text{obs}}_{s,t} = \min(y_{s,t},\ A_{s,t}) \tag{5}$$

where $A_{s,t}$ is the units available. Training on y^{obs} without correction is not merely biased — it is a closed feedback loop. The stockout depresses the forecast, the depressed forecast depresses the reorder quantity, the smaller order produces another stockout, and every symptom of the loop presents as an ordinary decline.

4.1 Detection

A bucket is flagged when end-of-bucket inventory is non-positive:

$$Z_{s,t} = \mathbf{1}\{ I_{s,t} \le 0 \} \tag{6}$$

4.2 Recovery

Unconstrained demand is estimated from the series' own uncensored behaviour: a trailing level times a day-of-week shape. Both estimators use only uncensored buckets, and the level is strictly backward-looking so that the estimate for a bucket never uses that bucket or any later one:

$$\lambda_{s,t} = \operatorname{median}\{ y_{s,u} : t - W \le u < t,\ Z_{s,u} = 0 \} \tag{7}$$

$$\varphi_{s,d} = \frac{\operatorname{median}\{ y_{s,u} : \operatorname{dow}(u) = d,\ Z_{s,u} = 0 \}}{\operatorname{median}\{ y_{s,u} : Z_{s,u} = 0 \}} \tag{8}$$

The recovered observation is then clamped from both sides:

$$\hat{y}_{s,t} = \min\left(\max\left(y^{\text{obs}}_{s,t},\ \lambda_{s,t}\, \varphi_{s,\operatorname{dow}(t)} \right),\ \kappa_{\max}\, \lambda_{s,t} \right) \tag{9}$$

The lower clamp encodes that units which did sell are a fact and only the unsold remainder is being inferred. The upper clamp, with $\kappa_{\max} = 3$, prevents a single mis-flagged bucket in a spiky series from being lifted into an outlier that then dominates the level estimate of everything downstream. Recovery is suppressed entirely for series with fewer than fourteen uncensored buckets, which is the point at which (8) ceases to be estimable — two observations per weekday are needed for the shape to exist at all, and below that the estimator silently degenerates to a bare median while still publishing its output as recovered demand.

4.3 The guard that matters more than the method

The canonical schema broadcasts *inventory* = 0 into every session in which the user did not map an inventory column. A naive reading of (6) would therefore flag every row of every such dataset as censored and inflate the entire tenant's forecast. The engine treats an inventory column that is never positive as absent evidence and returns the frame untouched. Declining to act is the correct failure mode here: the bias being corrected is real, but fabricating demand from evidence one does not have is worse than the bias.

5. Feature construction

Features fall into two classes with different leakage properties. Autoregressive features are functions of the target's own past and must be shifted; calendar features are deterministic functions of the date and are available for any bucket, past or future. Conflating the two is the origin of a large fraction of silent forecasting defects.

5.1 Autoregressive features

All of the following are computed within series and are functions of $\mathscr{E}_{s,t-1}$ only. The shift is applied *before* the window operator, not after:

Feature	Definition	Note
lag_ℓ	$y_{s,t-\ell}$	Direct autoregression.
roll_mean_w	$\frac{1}{w}\sum_{j=1}^{w} y_{s,t-j}$	Level over the trailing window.
roll_std_w	$\operatorname{sd}(y_{s,t-1}, \ldots, y_{s,t-w})$	Volatility.
roll_min_w / roll_max_w	$\min/\max_{j=1..w}\ y_{s,t-j}$	Range.
cv_w	$\text{roll\_std}_w / (\lvert\text{roll\_mean}_w\rvert + \varepsilon)$	Scale-free dispersion.

Feature	Definition	Note
ewm_α	$\sum_{j \geq 1} (1-\alpha)^{j-1} \alpha \, y_{s,t-j}$	Exponentially weighted level.
diff_d	$y_{s,t-1} - y_{s,t-1-d}$	Differenced on the shifted series.
pct_change_d	$(y_{s,t-1} - y_{s,t-1-d}) / (\lvert y_{s,t-1-d} \rvert + \varepsilon)$	Relative change.

Table 2. Autoregressive features. Every definition is indexed from t-1, never t.

The differencing entries deserve comment. An unshifted difference $y_t - y_{t-d}$ satisfies $y_t = \text{lag}_d + \text{diff}_d$ and therefore hands the model an exact algebraic reconstruction of its own target. The resulting validation error is near zero and the resulting forecast extrapolates the last observed slope indefinitely. Shifting first removes the identity.

5.2 Calendar and holiday features

Calendar features are generated by a single function used by both training and inference. This is an architectural constraint, not a convenience: any divergence between the two produces a feature that carries signal during fitting and a constant at serving time, which is strictly worse than omitting the feature. Alongside the usual cyclical encodings

$$\sin\left(\frac{2\pi m}{12}\right),\ \cos\left(\frac{2\pi m}{12}\right),\ \sin\left(\frac{2\pi \text{dow}}{7}\right),\ \cos\left(\frac{2\pi \text{dow}}{7}\right) \tag{10}$$

the engine emits holiday indicators and signed proximity. Let $\mathbb{H}$ be the national holiday set for the tenant's country, resolved from calendar rules and therefore defined for future years as readily as past ones. Easter is computed in closed form by the anonymous Gregorian algorithm, guaranteeing a non-empty holiday set even where a national calendar is unavailable:

$$\delta_t = \min_{\tau \in \mathbb{H}} |t - \tau| \tag{11}$$

Proximity is evaluated by binary search over the sorted holiday ordinals, giving $O(n \log |\mathbb{H}|)$ rather than the $O(n \cdot |\mathbb{H}|)$ of a naive scan. A composite intensity score weights the classes of holiday that move retail demand by materially different amounts.

5.3 Fourier seasonality on a fixed epoch

Fourier terms of period P and order K supply smooth seasonality that survives sparse data:

$$\left\{ \sin\left(\frac{2\pi k\, \tau_t}{P}\right),\ \cos\left(\frac{2\pi k\, \tau_t}{P}\right) \right\}_{k=1}^{K}, \qquad \tau_t = t - t_0 \tag{12}$$

The choice of origin t_0 is not free. If t_0 is the first date of the uploaded file, the phase of every term depends on the customer's export window, two uploads covering the same calendar day disagree about that day's features, and inference cannot reproduce a value it never saw the origin for. The engine fixes t_0 at the Unix epoch so that the map from date to feature is global and reproducible.

6. Model families

The engine trains a portfolio and selects per series. Classical local models remain valuable on long, well-behaved series; the global model exists for the majority of a real catalogue, which is neither.

6.1 Local per-series models

Family	Form	Suited to
ARIMA / SARIMAX	$\phi(B)(1-B)^d y_t = \theta(B)\varepsilon_t \; (+\beta^\top x_t)$	Long, stationary-after-differencing series; exogenous regressors.
ETS	$\ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1})$	Smooth trend and stable seasonality.
Croston / TSB	$\hat{y} = \hat{z}/\hat{p}$	Intermittent demand: size and interval estimated separately.
Prophet	$y_t = g(t) + s(t) + h(t) + \varepsilon_t$	Strong multi-scale seasonality with changepoints.
GBDT (LightGBM, XGBoost)	$F_M(x) = \sum_{m=1}^{M} \nu f_m(x)$	Nonlinear interaction of engineered features.
LSTM	$h_t = \mathrm{LSTM}(x_t, h_{t-1})$	Long nonlinear dependence; requires substantial history.

Table 3. Local model families. Each is fitted independently per series.

A router assigns each series a candidate subset from a classification of its own statistics — zero ratio, coefficient of variation, and STL seasonal strength. Routing only ever *narrows* the user's selection; it can never introduce a model the user did not choose.

6.2 The global cross-learning model

Fitting an independent model per series is the wrong estimator for this regime. A catalogue of three thousand SKUs with fourteen months of history yields three thousand estimation problems, most of them with a few dozen effective observations, each blind to the fact that the SKU beside it is the same product in another size and peaks in the same week. Series below a minimum history are routed to naive baselines and a newly-launched SKU receives nothing at all.

The global model replaces this with a single estimation problem over the pooled catalogue. Two devices make pooling informative rather than merely averaging.

6.2.1 Per-series normalisation

Raw pooling lets a SKU selling 5000 units a day dominate the squared loss of one selling 5. The target and every feature denominated in units are divided by a series-specific scale:

$$\mu_s = \max(|\mathrm{mean}\{y_{s,t} : t \in W_0\}|,\ \varepsilon), \qquad \tilde{y}_{s,t} = y_{s,t}/\mu_s \tag{13}$$

The calibration window W_0 ends at the first cross-validation cutoff, so the scale never sees a bucket it will later be evaluated on. Features are partitioned by dimensional analysis: quantities in units of the target (lags, rolling statistics, exponentially weighted means, differences) are divided by μ_s; dimensionless ones (coefficients of variation, relative changes, calendar terms) are left alone. Dividing a ratio by a scale would be a unit error, and it would destroy the feature.

Normalisation alone would erase the information that a series is large or small, which is itself predictive. The scale is therefore reintroduced as an explicit covariate together with a dispersion summary:

$$\ell_s = \log(1+\mu_s), \qquad v_s = \mathrm{sd}\{y_{s,t} : t \in W_0\}/\mu_s \tag{14}$$

6.2.2 Series identity

Each identity dimension is encoded as an integer code and declared categorical to the booster, which splits on subsets of levels rather than on an arbitrary ordinal ordering. The model can therefore specialise toward a particular series where the evidence supports it and fall back on the pooled structure where it does not — the shrinkage is learned rather than imposed.

6.2.3 Direct multi-horizon formulation

The engine does not forecast recursively. Recursion feeds a prediction back as though it were an observation, so a fourteen-step forecast is built on thirteen guesses, errors compound multiplicatively through the lag features, and the intervals — derived from one-step residuals — fail to widen to match. Instead the horizon is a feature. Each training row pairs the features known at an origin with the realised value h buckets later:

$$\mathcal{D} = \{(\tilde{x}_{s,i},\ c_s,\ \ell_s,\ v_s,\ h) \longmapsto \tilde{y}_{s,\,i+h-1}\}_{s\in S,\ i,\ h=1..H+1} \tag{15}$$

The model is the gradient-boosted regressor minimising squared error on the normalised target,

$$\hat{\theta} = \arg\min_{\theta} \sum_{(\cdot)\in D} (\tilde{y}_{s,\,i+h-1} - f_\theta(\tilde{x}_{s,i}, c_s, \ell_s, v_s, h))^2 \tag{16}$$

and the forecast for future step k is a single direct evaluation, rescaled and truncated at zero:

$$\hat{y}_{s,\,T+k} = \mu_s \cdot \max(0,\ f_{\hat{\theta}}(\tilde{x}_{s,\,\text{origin}},\ c_s,\ \ell_s,\ v_s,\ h = g_s + k)) \tag{17}$$

The offset g_s in (17) is not cosmetic. Because every autoregressive feature is indexed from $t-1$, the freshest constructible feature row conditions on the *second*-to-last observation, so the first genuinely future bucket lies $g_s + 1$ steps from the origin with $g_s = 1$. Omitting the offset publishes a forecast of the last observed bucket as though it were the first future one: aggregate error statistics look entirely normal and every dated value is wrong by one bucket. The stacked design in (15) is built to $H+1$ precisely so that the final step remains inside the horizon range the model was fitted on, rather than extrapolating the horizon covariate.

Cost. The stacked design has $|\mathcal{D}| = O(N{\cdot}H)$ rows for N origins. Above a fixed ceiling the engine thins *origins* by a stride and never *horizons*: dropping origins costs sample size, whereas dropping horizons would leave entire steps of the published forecast with no training signal at all.

7. Validation

7.1 Walk-forward with a withheld gap

Local models are validated by expanding-window cross-validation. For fold i with test window opening at τ_i:

$$\mathcal{T}_i^{\text{train}} = \{1,\ \ldots,\ \tau_i - g\}, \qquad \mathcal{T}_i^{\text{test}} = \{\tau_i,\ \ldots,\ \tau_i + m\} \tag{18}$$

with g set to the forecast horizon. Without the gap the model is fitted on data ending the bucket before the one it is graded on, while the freshest observation any production forecast can hold is h buckets old; for an autocorrelated series that is the most informative data there is, and the resulting score describes an easier problem than the product solves. When a short series cannot fund the full gap it is reduced rather than abandoned, subject to preserving both the fold count and a minimum training width — a fold of two rows is not a fold, merely a shape that does not raise.

It is worth stating what the gap does *not* fix. Test rows still carry their own true lag features, so each row is still graded as a one-step problem. Only a model conditioning on a fixed origin, as in (17), is scored honestly across the whole horizon.

7.2 Rolling-origin backtest

The global model is validated by simulating the production act. At each cutoff T_i the model is refitted on rows whose *target* date precedes the cutoff — not merely whose feature date does, which is the subtler of the two leaks — and is then asked to forecast forward from the last observation:

$$r_{s,h}^{(i)} = \tilde{y}_{s,\,T_i+h} - \hat{\tilde{y}}_{s,\,T_i+h\,|\,T_i} \tag{19}$$

Folds run over the *date* axis rather than over row positions. With thousands of series of differing lengths concatenated, a row index is meaningless and splitting on it would place one SKU's future inside another SKU's past.

8. Uncertainty quantification

8.1 What the Gaussian band assumed

The conventional construction is

$$\hat{y}_{T+h} \pm z_\alpha \hat{\sigma}, \qquad \hat{\sigma} = \mathrm{sd}(\{r_t\}) \tag{20}$$

with σ estimated from one-step residuals and reused unchanged for every step. Three assumptions are compounded there, and retail demand violates all three: that the error is Gaussian, when demand is non-negative, discrete and frequently zero; that it is symmetric, which a floor at zero precludes; and that a fourteen-step error is the size of a one-step error, which it is not. The band is therefore narrowest in exactly the region where the buyer most needs it wide.

8.2 Split conformal calibration per horizon

The engine replaces (20) with empirical quantiles of the realised backtest residuals, computed separately for each horizon. Pooling the residuals of (19) across the catalogue in *normalised* units gives

$$\hat{q}_h(\alpha) = \mathrm{Quantile}_\alpha\left(\left\{r_{s,h}^{(i)}\right\}_{i,\,s}\right) \tag{21}$$

$$C_\alpha(\hat{y}_{s,\,T+h}) = \hat{y}_{s,\,T+h} + \mu_s\,\hat{q}_h(\alpha) \tag{22}$$

No distributional assumption is made, the band widens with h because the residuals do, and asymmetry is preserved rather than averaged away. Normalisation is what makes the pooling legitimate and is also what makes it valuable: a SKU with eight weeks of history inherits the shape of the catalogue's error distribution instead of estimating its own from a handful of points. Where a horizon has too few residuals to support a quantile, the engine falls back to the pooled bank across horizons and says so, rather than reporting a confident number computed from four observations. Independently estimated quantiles can cross; a running maximum restores monotonicity while preserving every quantile that was already consistent.

Coverage is finite-sample valid under exchangeability of the residuals. Demand residuals are not perfectly exchangeable across time, so the guarantee should be read as strong-in-practice rather than exact — which is still a materially stronger claim than (20) can make.

8.3 Cumulative bands for the lead time

Section 1 established that the decision-relevant variable is a sum. The same backtest yields its error directly:

$$R_{s,L}^{\mathrm{cum}} = \sum_{h=1}^{L}\left(\tilde{y}_{s,\,T+h} - \hat{\tilde{y}}_{s,\,T+h}\right) \tag{23}$$

$$\widehat{Q}_\beta[D_{s,\,T}(L)] = \sum_{h=1}^{L}\hat{y}_{s,\,T+h} + \mu_s\,\hat{q}_L^{\mathrm{cum}}(\beta) \tag{24}$$

Equation (24) is the quantity the reorder point needs, obtained without assuming independence across buckets. The classical alternative, $\sigma\sqrt{L}$, is exactly the independence assumption in disguise, and it understates risk precisely on the SKUs whose forecast bias is persistent — a forecast that runs high today usually runs high tomorrow.

9. Model selection under an asymmetric loss

Candidate models were previously ranked by MAE, and the displayed accuracy by WAPE. Both are symmetric: they score a forecast ten units low exactly as well as one ten units high. For a distributor the two are not equivalent — the surplus costs warehouse space and working capital, the shortfall costs the sale and sometimes the customer. The engine ranks by an asymmetric per-unit cost,

$$\mathcal{L}_{\kappa}(y,\hat{y}) = \tfrac{1}{n}\sum_{t=1}^{n}\left[(\hat{y}_t - y_t)_+ + \kappa(y_t - \hat{y}_t)_+\right] \tag{25}$$

with κ the stockout-to-holding ratio. Where a candidate produces a quantile forecast rather than a point, the proper scoring rule is the pinball loss, which is the loss the reorder point is actually optimal for:

$$\mathcal{P}_q(y,\hat{y}) = \tfrac{1}{n}\sum_{t=1}^{n}\max\left(q\,(y_t - \hat{y}_t),\ (q-1)(y_t - \hat{y}_t)\right) \tag{26}$$

Note that scoring a $q = 0.95$ forecast with MAE rewards it for being close to the middle of the distribution, which is precisely what a reorder point must not be.

Two honesty constraints follow. First, the accuracy reported to the user is the WAPE *of the model actually selected*, not the best WAPE available for that series; those coincided while selection was by WAPE and diverge once it is by cost, and reporting the better figure would advertise a forecast nobody is using. Second, baselines are scored and displayed but excluded from selection: they exist to be beaten, and purchasing from a naive forecast because it happened to win a fold is a defect, not a fallback.

10. From predictive distribution to purchase order

10.1 Reorder point and safety stock

Given (24), the reorder point and the implied cushion are read directly off the measured distribution:

$$\mathrm{SS}_s = \mu_s\,\hat{q}^{\mathrm{cum}}_L(\beta), \qquad \mathrm{ROP}_s = \sum_{h=1}^{L}\hat{y}_{s,T+h} + \mathrm{SS}_s \tag{27}$$

Where the measurement is unavailable the engine falls back explicitly to the classical form,

$$\mathrm{SS}_s^{\mathrm{classical}} = z_\beta\,\hat{\sigma}_s\,\sqrt{L} \tag{28}$$

and the two must never be composed. A recurring class of defect is to read the top of an existing band as though it were a standard deviation and then multiply by z again; since the band was itself approximately a 90th percentile, the result is a configured 95% service level being served at roughly 98% — more capital tied up than the buyer asked for, with nothing in the product disclosing the discrepancy. Both branches of the engine's estimator return the same quantity, a standard deviation, by dividing any band by the z that produced it.

The order quantity applies on-hand stock and the supplier's minimum:

$$Q_s = \left\lceil \frac{\max(0,\ \mathrm{ROP}_s - \mathrm{stock}_s)}{\mathrm{MOQ}_s} \right\rceil \cdot \mathrm{MOQ}_s \tag{29}$$

The service level itself need not be a free parameter: under linear holding and shortage costs the newsvendor optimum is $\beta^* = c_u / (c_u + c_o)$, so a tenant who can state its cost ratio can derive the level rather than guess it.

10.2 Multi-warehouse allocation

With several locations, purchasing and inter-warehouse transfer are solved jointly as a mixed-integer program over buckets $t = 1..H$. Writing o, x, v, u for order, transfer, ending inventory and shortage, the balance constraint is

$$v_{i,w,t} - v_{i,w,t-1} - o_{i,w,t} - \sum_{a \neq w} x_{i,a,w,t} + \sum_{b \neq w} x_{i,w,b,t} - u_{i,w,t} = -d_{i,w,t} \tag{30}$$

and the objective minimises total cost,

$$\min \sum_{i,w,t} \left(c_i^h v_{i,w,t} + c_i^u u_{i,w,t} + c_i^o o_{i,w,t} \right) + \sum_{a,b,t} \left(c_{ab}^x x_{\cdot,a,b,t} + c_{ab}^f \zeta_{a,b,t} \right) \tag{31}$$

where ζ is a binary indicating that lane (a,b) dispatches in bucket t, linked by $x \leq M_i \zeta$. The big-M is derived per SKU from that SKU's own stock plus horizon demand rather than from a global constant: a single M dominated by the largest SKU in the catalogue drives the ratio x/M for every other SKU below the solver's integrality tolerance, at which point the fixed cost is satisfied at $\zeta \approx 0$ and shipments ride for free. Per-SKU scaling also tightens the linear relaxation. Lead times enter as upper bounds of zero on arrivals earlier than the transit time allows.

11. Hierarchical reconciliation

Forecasts produced independently at SKU, category and total level are mutually inconsistent. With S the summing matrix mapping bottom-level series to all levels, reconciliation projects the base forecasts onto the coherent subspace:

$$\tilde{y} = S(S^\top W^{-1} S)^{-1} S^\top W^{-1} \hat{y} \tag{32}$$

Bottom-up and top-down are the special cases in which W is chosen to select a single level. The minimum-trace solution instead sets W to the covariance of the base forecast errors, usually via a shrinkage estimator toward a diagonal target. The practical consequence is that MinT improves the bottom level as well as the aggregate, whereas bottom-up leaves the SKU forecasts exactly as they were and top-down discards their individual information entirely.

12. Limitations

- **Cross-family comparability.** Local ML models are graded one step ahead, statistical models across their whole test window, and the global model on a rolling origin. These are different questions. Selection is fairer than ranking by symmetric error, but it is not yet apples to apples across families; unifying the protocol is the single largest remaining correctness gain.
- **Censoring evidence.** Recovery requires historical inventory. Tenants who upload sales alone keep the downward bias of Section 4, and the engine cannot detect that they have it.
- **Exchangeability.** Conformal coverage assumes exchangeable residuals. A structural break — a new distribution channel, a price reposition — violates it precisely when the bands matter most.
- **Substitution and cannibalisation.** Series are modelled as conditionally independent given their features. Demand transfer between substitutable SKUs, and the demand a stockout displaces onto a neighbour, are outside the current formulation.
- **Fixed cost asymmetry in ranking.** Selection uses a standard 3:1 shortfall-to-surplus ratio for every tenant; the tenant's configured ratio drives the order quantity but not the ranking.
- **Price and promotion.** The schema carries them and SARIMAX consumes them as exogenous regressors, but the global model does not yet treat a planned future price as a known future covariate, which is where most of the remaining promotional signal lives.

13. Summary of the estimator

Stage	Choice	Rejected alternative
Pooling	One model across the catalogue, per-series normalised	Independent model per series
Horizon	Direct, horizon as a covariate	Recursive substitution of predictions

Stage	Choice	Rejected alternative
Identity	Categorical codes + level + dispersion covariates	Series-agnostic pooling
Validation	Rolling origin on the date axis, gap = horizon	Row-index folds with adjacent train/test
Intervals	Split conformal, per horizon, pooled in scaled units	Gaussian band from one-step residuals
Lead-time risk	Measured cumulative quantile	$z_\beta \sigma \sqrt{L}$
Selection	Asymmetric cost / pinball loss	MAE or WAPE
Censoring	Detect from inventory, lift with clamps	Train on observed sales

Table 4. The engine's design choices, each against the convention it replaces.

References

Hyndman, R. J., & Athanasopoulos, G. *Forecasting: Principles and Practice*, 3rd ed. OTexts.

Wickramasuriya, S. L., Athanasopoulos, G., & Hyndman, R. J. (2019). Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization. *JASA*, 114(526).

Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.

Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L. (2018). Distribution-free predictive inference for regression. *JASA*, 113(523).

Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS*.

Januschowski, T., et al. (2020). Criteria for classifying forecasting methods. *International Journal of Forecasting*, 36(1).

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings and conclusions. *International Journal of Forecasting*, 38(4).

Syntetos, A. A., & Boylan, J. E. (2005). The accuracy of intermittent demand estimates. *International Journal of Forecasting*, 21(2).

Nahmias, S., & Olsen, T. L. *Production and Operations Analysis*, 7th ed. Waveland Press.